Still Needed, Hard to Transfer: Using and Reusing Contextual Skills

Anonymous Author(s)

Abstract

Contextual skills supply reusable instructions to language-model agents, motivating efforts to transfer the resulting internal representations across executions. When such transfer weakens at later layers, does the original execution also depend less on the skill text, or does the transfer omit components that still matter? We pair skill-position residual replacement in a wrong-skill receiver with cumulative skill-attention knockout in the correct-skill execution, comparing both interventions on common eligible questions across depth. Early content and position controls validate the transfer probe, and a focused experiment adds skill-position key/value cache replacement. Across eight verified model–benchmark comparisons involving four open-weight models on MedCalc-Bench and TheoremQA, transfer recovery falls below half of the donor's gain before skill-attention dependence falls to half, revealing a depth interval with weak transfer but persistent dependence. On Qwen3-8B/MedCalc at layer 16, adding skill-cache replacement raises accuracy from 3.5% to 42.1%, while a subsequent post-skill-to-skill cut reduces it to 7.0%; the same layer-16 comparison yields a 6.8–43.4-point gain across all eight settings. This supports partial repair without uniquely explaining late transfer failure.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Knowledge representation and reasoning*.

Keywords

agent skills, Web agents, large language models, in-context learning, activation patching, context reuse, mechanistic interpretability

1 Introduction

Relevance to the Web and to this track. Web agents rely on machine-readable instructions and descriptions of tools and services to understand how to act across sites [24, 25]. Agent Skills package such procedural knowledge for reuse. We ask how a model uses a loaded skill and when the internal states it produces can support another execution. This connects our study to the Semantics and Knowledge track's interest in machine-readable descriptions that help agents understand and use Web resources, and in knowledge representations that support intelligent agent behaviour.

Agent Skills package instructions, scripts and resources around a `SKILL.md` entry point [28]. We study the instruction content after the skill has been selected. For example, a calculator skill supplies a formula, input conventions and worked steps; the model must apply them to a new question. Web agents similarly use machine-readable descriptions of tools and services [24, 25]. Understanding the limits of reusing their processed representations connects this study to the Web's semantics and knowledge infrastructure. Our experiments test controlled skill-based question answering, rather than interactive browsing.

Task and function vectors show that internal states can transmit task behaviour across contexts [12, 22]. However, restoring a corrupted execution and disrupting a successful one measure different intervention effects; one need not determine the other [11]. Our contribution is to measure how those effects separate across depth for supplied skills, and to test a specific component omitted by residual-state transfer.

We first validate a transfer probe that copies hidden vectors at the skill's token positions from a correct-skill donor into an equal-length wrong-skill receiver. Content, position and cross-question controls test whether its early benefit carries structured skill information. We then compare this probe across all layers with cumulative skill-attention knockout in the original correct-skill execution. This knockout blocks both access from post-skill tokens and attention within the skill text. It therefore measures aggregate skill-attention dependence, not post-skill-only dependence.

Figure 1 previews the two levels of evidence. The main finding is a depth interval in which residual transfer is weak while skill dependence remains high. The ordering of their first half-effect crossings recurs in all eight comparisons across four main models, using common eligible question IDs within each comparison; a smaller-model scale check is reported in Appendix B. These are conditional effects on screened populations, and matching IDs does not make the donor and receiver execution backgrounds identical.

To probe one possible contributor to the late transfer deficit, we replace the skill-position key/value (KV) caches that residual replacement leaves unchanged. In a focused Qwen3-8B/MedCalc experiment, joint state and cache replacement raises layer-16 accuracy from 3.5% to 42.1%; a subsequent post-skill-to-skill cut reduces it to 7.0%. This identifies a partial repair, not a complete account of the failure. The intervention changes which skill-position caches are available across layers, does not replay earlier post-skill computation, and remains far below the donor's 100% accuracy on this freshly screened set. The same layer-16 comparison shows positive repair and recutting effects across all eight main model–task settings, with different effect sizes and independent baseline screens. Distinguishing cache coverage from more specific state–history interactions requires additional controls.

The experiments follow this evidential progression: validate the early probe (§4.2), establish the depth separation (§4.3), and measure the effect and limits of partial cache repair (§4.4). The resulting contribution is a replicated depth ordering under the tested transfer and knockout protocols, together with a localized skill-cache intervention that recovers part of late transfer. It does not establish that the skill is fully internalized at a particular layer or that cache mismatch uniquely explains weak transfer.

WWW '27, Dublin, Ireland
2027.

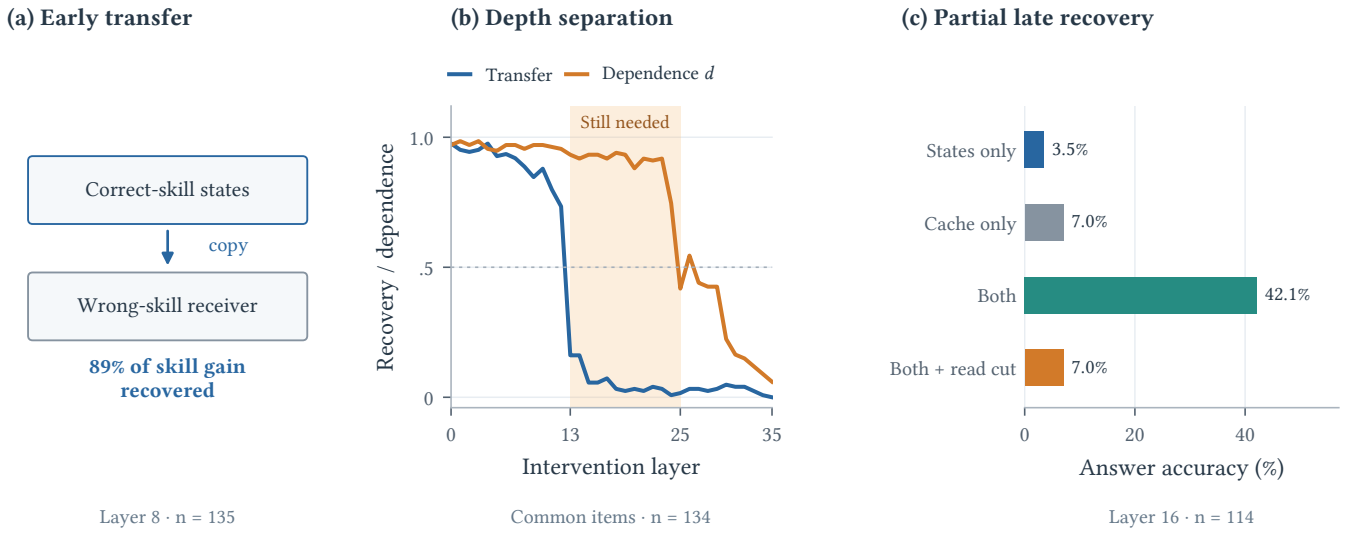


Figure 1: A depth separation and a partial repair. (a) Early residual replacement recovers most of the re-evaluated donor's gain on a historically screened Qwen3-8B/MedCalc population. (b) Residual transfer weakens while aggregate skill-attention dependence $d = 1 - q$ remains high on common eligible items. (c) At layer 16, adding skill-cache replacement partially restores transfer; a subsequent post-skill-to-skill cut removes most of that gain. The panels use distinct evaluation populations, indicated below each panel.

2 Related Work

State transfer and its interpretation. Task vectors summarize demonstrations as internal vectors [12], and function vectors identify attention-head outputs that transmit task behaviour [22]. Instructions also yield transferable components [2, 5], whose effects vary with intervention depth [17]. Activation patching requires matched controls and careful implementation [11, 29]. In particular, restoring behaviour and disrupting it need not reveal the same components. Interactions with other components can also affect a patching estimate [23]. We build on these methodological distinctions; the general difference between restoration and disruption is not itself our empirical contribution.

Depth, skill access and context caches. Pola and Balasubramanian [19] locate an onset beyond which selected residual patches largely cease to change answers in their instruction tasks. Sia et al. [20] use layer-wise context masking to test when translation no longer requires attention to its context. Joint transfer and masking analyses also locate subtask computation [27]. We compare residual transfer with skill-attention knockout within skill-based settings, then test an omitted skill-position component. Differences in tasks, models and interventions prevent our results from directly refuting the earlier onset or translation findings. PrefillNotes [15] studies reusable skill caches. Our focused cache intervention asks how extending skill replacement affects an observed late transfer deficit; it is not a full-cache reuse system or a unique test of history matching.

Skills and reusable context. Agent Skills organize instructions and resources for on-demand use [28]. Benchmarks study skill retrieval and application [10, 16, 21], while matched-budget evaluations expose their dependence on available computation [9]. We hold skill selection fixed. Other approaches compress context into aggregated activations [1], learned summary tokens [18], hidden states [7], or weight updates [3, 4]. Our interventions characterize naturally occurring states and skill dependence without training a compression mechanism.

3 Problem Formulation

Setting and notation. We supply a skill's instructions directly and hold skill selection fixed. For example, the released BMI skill specifies weight in kilograms, height in metres and division by squared height, alongside conversion rules. The skill span is this instruction text, not the patient's question. For the same question, the *donor* uses the correct skill and the *receiver* an unrelated skill of equal token length. Let $\mathcal{I}$ be the token positions occupied by the skill text, the *skill span*. We call positions after this span *post-skill positions*: these include subsequent prompt tokens (such as the question) and generated reasoning and answer tokens. These are token roles within each execution, distinct from the donor and receiver executions. Tokens before the skill cannot attend to it under causal attention. Layers are indexed from zero. The model processes prompt tokens during *prefill*, then generates answer tokens during *decoding*. We compare transferring skill states into the receiver with removing skill access in the donor (Figure 2).

Residual-state and cache replacement. Let H^L be the residual states, the hidden vectors output by block L. Subscripts D and R denote donor and receiver. State replacement, labelled S, writes

$$S: \qquad H_R^L[\mathcal{I}] \leftarrow H_D^L[\mathcal{I}]. \tag{1}$$

This write occurs during prefill; the receiver then continues normally. Other positions at that layer are unchanged. The intervention

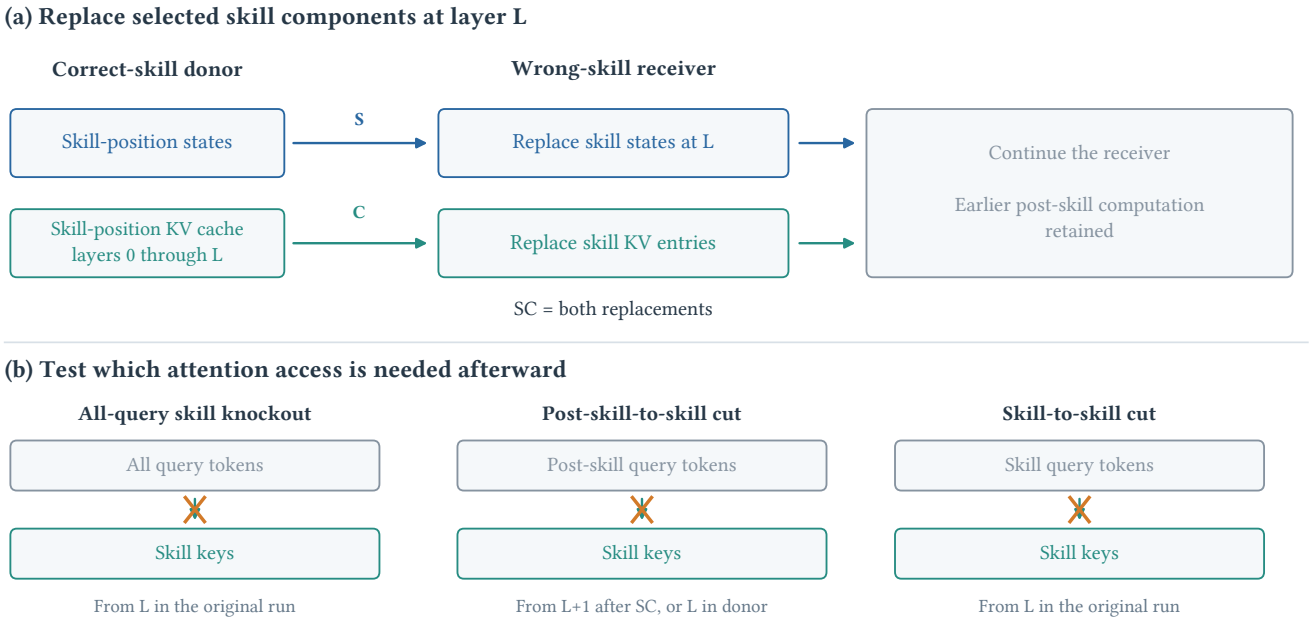


Figure 2: Intervention design. (a) The correct-skill donor supplies skill residual states (S), skill-position key/value (KV) entries through layer L (C), or both (SC) to the wrong-skill receiver. Earlier computation at post-skill positions is retained. (b) Attention cuts distinguish access by post-skill tokens from updates within the skill text. After SC, the post-skill-to-skill cut begins at $L+1$. In the original correct-skill run, path cuts begin at L. All cuts continue through the remaining prompt processing and decoding.

repairs an execution that has processed an unrelated skill, so its effect can include both supplying useful information and overcoming that receiver background.

Attention matches a query to keys and combines their associated values. The key/value (KV) cache stores these vectors for processed tokens. Let K^ℓ and V^ℓ be the cached keys and values at layer ℓ. Cache replacement, labelled C, writes

$$C: \quad \begin{aligned} K_R^\ell[\mathcal{I}] &\leftarrow K_D^\ell[\mathcal{I}], \\ V_R^\ell[\mathcal{I}] &\leftarrow V_D^\ell[\mathcal{I}], \end{aligned} \qquad 0 \le \ell \le L. \tag{2}$$

The combined intervention SC performs both writes. S alone leaves these earlier caches unchanged; neither write replays earlier post-skill computation. Cache writes precede processing the final prompt token (Appendix A.1). S can change later skill states and their subsequently computed caches, whereas C writes the skill-position caches through L. Their combination thus also changes cache coverage across depth. It is not a controlled comparison between matched and mismatched histories with coverage held fixed.

Cumulative attention knockout. Let z_{ij}^ℓ be the attention score from query position i to key position j, including the causal mask that blocks future tokens. At every layer $\ell \ge L$, knockout replaces these scores by

$$\tilde{z}_{ij}^\ell = \begin{cases} -\infty, & j \in \mathcal{I}, \\ z_{ij}^\ell, & j \notin \mathcal{I}. \end{cases} \tag{3}$$

Cuts block query-to-key connections; they do not delete skill tokens or erase information already computed at other positions. Softmax converts the modified scores into attention weights: blocked positions receive zero weight and the remaining weights sum to one. The cut applies to every query and attention head (a parallel attention component), during both prefill and decoding. It affects access from post-skill tokens and skill-state updates. Path-specific cuts restrict i to post-skill positions or to $\mathcal{I}$; a post-skill-to-skill cut after SC starts at $L+1$. We reserve *post-skill-to-skill dependence* for these path-specific tests. The all-query skill knockout alone cannot distinguish blocked reading from disrupted skill-state updates, or prefill effects from decoding effects.

Recovery and dependence. Write a_D, a_R and a_I for answer accuracy under the donor, receiver and a specified intervention I. Recovery measures the fraction of the donor's gain restored:

$$\rho(I) = \frac{a_I - a_R}{a_D - a_R}. \tag{4}$$

Thus $\rho = 1$ restores that gain; $\rho > 1$ exceeds donor accuracy. For state replacement at layer L, we write $\rho(L)$. All three accuracies use the same items and the transfer run's own unpatched baselines. Candidate selection is a separate step: the original transfer battery starts from *historically rescued* items, selected by earlier without-skill and with-skill evaluations. Their donor need not remain correct when re-evaluated in the intervention protocol. The group screen retains skills whose re-evaluated receiver solves no sampled item. Section 4.1 distinguishes this population from freshly baseline-screened knockout and mechanism populations.

On the knockout's eligible items, let $q(L)$ be the fraction still correct. Their *skill-attention dependence* is

$$d(L) = 1 - q(L). \tag{5}$$

It is the fraction of originally correct answers lost under knockout. Higher values mean stronger dependence on subsequent skill attention. We also compare both curves on common eligible items, retaining their own baselines and distinct donor/receiver execution backgrounds. These are two population-level effects, not complementary probabilities or the fraction of individual questions that fail both interventions.

Depth boundaries. After establishing effective early transfer, define the first half-effect boundaries and their separation as

$$\begin{aligned} T &= \min\{L : \rho(L) < 0.5\}, \\ R &= \min\{L : d(L) \le 0.5\}, \qquad \Delta = R - T. \end{aligned} \tag{6}$$

Layers with $\rho(L) < 0.5$ and $d(L) > 0.5$ show weak transfer despite strong dependence. Figure 3 shades the interval between the first boundaries and preserves later recrossings. The half-effect thresholds summarize effect size; T is not complete transfer failure, and R is not the end of skill dependence. Layers are intervention depths, not successive times in answer generation: every generated token passes through the layer stack again.

4 Experiments

We first establish an effective early transfer probe, then compare its depth profile with aggregate skill dependence. The final experiment asks whether replacing skill-position caches omitted by that probe can repair part of the late deficit. Each step supports a different level of inference.

4.1 Experimental setup

Tasks and models. We pair MedCalc-Bench [14] with skills from SRA-Bench [21], covering 1,100 questions from 55 clinical calculators. We also use SRA-Bench's TheoremQA subset, with 747 questions and 320 skills. Each question receives its designated skill, holding skill selection fixed. The main comparison uses four open-weight models from four families: Qwen3-8B [26], Mistral-7B-Instruct-v0.3 [13], Llama-3.1-8B-Instruct [8] and DeepSeek-LLM-7B-Chat [6]. Qwen3-0.6B provides a supplementary scale check (Appendix B). Models generate step-by-step answers with greedy decoding, selecting the highest-probability next token at each step. We use no tools and apply the corresponding benchmark scorers. On the full MedCalc benchmark, Qwen3-8B achieves 74.2% accuracy with the correct skill, 34.5% without a skill, and 31.5% with an unrelated skill.

Selection and evaluation baselines. The original MedCalc/Qwen3-8B transfer battery begins with a historical behavioural screen: 467 questions from 48 calculators were incorrect without a skill and correct with it in that earlier evaluation. The intervention protocol re-evaluates donor and receiver baselines. The group-level receiver screen retains 135 questions from 14 calculators, without additionally requiring the re-evaluated donor to be correct on each question. Consequently, Table 1 has 124/135 correct donor answers, not 135/135. Its layer-8 transfer recovery is $(110/135-0)/(124/135-0) = 110/124$, approximately 0.89. Eligibility in the historical screen must not be confused with correctness under the transfer protocol.

For the depth comparison, we intersect transfer IDs with questions that are correct with and incorrect without the skill in the knockout run. This leaves 134 Qwen3-8B/MedCalc items; transfer retains its own re-evaluated baselines. The original transfer and knockout protocols use different attention backends, so common IDs control population membership without establishing protocol equivalence. The mechanism study instead starts from 124 fixed historical IDs and applies fresh baselines under one split-prefill protocol, retaining 114 items. Appendix A details these populations.

The added families use frozen candidate pools of 446 MedCalc and 255 TheoremQA questions inherited from the earlier models' eligible transfer items. Each new model is screened on its own unpatched baselines, with matched eager-attention protocols for transfer and knockout. One DeepSeek/MedCalc candidate is excluded in advance because its input cannot reserve the 900-token answer budget. These inherited pools and the wrong-skill group screen define conditional populations, rather than representative samples of either benchmark.

Implementation checks. An identity patch writes the receiver's own states back and should reproduce its output. Correcting an attention-mask mismatch between state capture and decoding restores exact agreement, including all 467 MedCalc identity checks. All main results use the corrected implementation. TheoremQA outputs for all five model sizes are regraded with the official evaluation rules while restoring fresh parser state before every answer. We retain the original text, verify grading-order invariance and compare with fresh-process evaluations; eligibility is recomputed before merging the depth supplements. Appendix A.1 specifies the intervention protocol, and Appendix A.2 records the correction.

4.2 Validation of skill-state transfer

Early transfer recovery. Before comparing transfer across depth, we establish that the intervention transmits useful behaviour. At layer 8 on the historically screened MedCalc set, replacing states at the skill's token positions recovers most of the correct skill's benefit (Table 1). Patching the last prompt position or the final 16 positions has a much smaller effect. This locates an effective intervention in the skill span. Its larger number of replaced states also makes it a higher-capacity intervention than the final-position alternatives.

Content and position controls. The unrelated-skill, permuted-position and noise controls in Table 1 all yield little recovery. The noise is scaled to match the size, measured by Euclidean norm, of the skill-state change. For the content comparison, the correct and unrelated skills are patched over a common prefix length. The correct prefix recovers 0.65 of the available gain. A mean of same-family skill differences recovers 0.12 on its smaller eligible set of 41 items. Rescaling patched states to match the receiver's norm at each position preserves 0.87 recovery, close to the full-span result. These comparisons identify skill-specific content and its arrangement as important to early transfer. The group screen matters: other-skill and noise controls are more effective in excluded groups, so the control contrast describes the screened population.

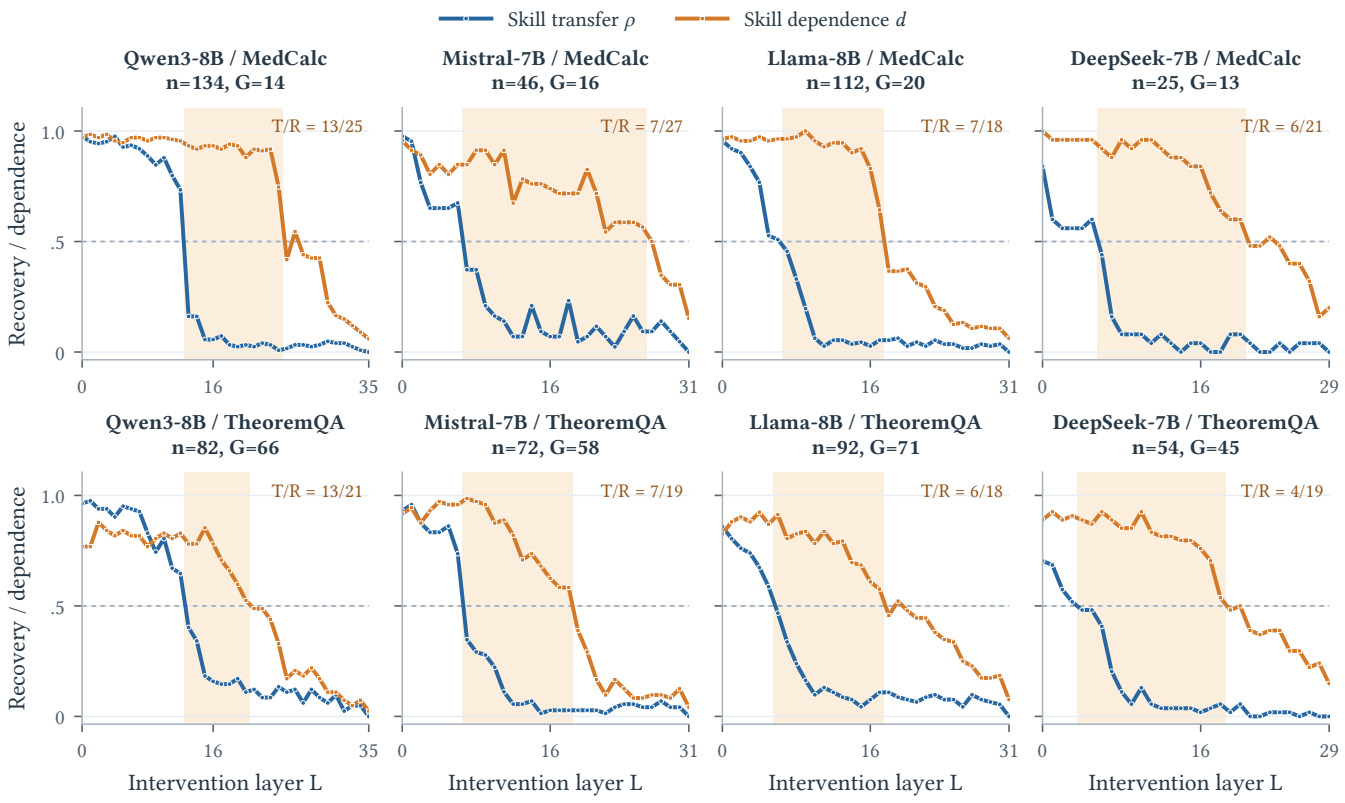


Figure 3: Depth profiles for all eight main model–benchmark comparisons. Blue shows skill-span transfer recovery $\rho(L)$ in a wrong-skill receiver. Orange shows aggregate skill-attention dependence $d(L) = 1 - q(L)$: the fraction of originally correct answers lost after cumulative attention knockout in the correct-skill execution. Shading spans the first half-effect boundaries, highlighting weak transfer despite strong dependence; later recrossings remain visible. Boundary layers are marked above each panel. Each panel uses common eligible item IDs for both curves, with each run's own baselines and execution protocol; G denotes skill groups. The knockout affects both access from post-skill tokens and skill-state updates. Every plotted layer is measured.

Cross-question transfer. In the main prompt order, the skill precedes the question. Causal attention, which restricts a token to attending to earlier positions, prevents those skill states from encoding the later question. We additionally place the skill after the question and obtain donor states from other questions using the same skill. On 151 eligible items from 17 calculators, these states recover 0.98 at layer 8, close to 1.02 for a donor answering the current question. At layer 16, the values separate to 0.29 and 0.49. Early transfer therefore carries information shared across instances, with greater question sensitivity later. Additional content, dose and rank diagnostics are reported in Appendix B; their role is to validate the probe rather than supply a separate mechanism for the depth separation.

4.3 Weak transfer with persistent skill dependence

Transfer–dependence separation. An effective early probe makes a weak later patch interpretable, but does not establish that the original run has stopped using the skill. Figure 3 therefore pairs both effects at every layer. On the 134 common Qwen3-8B/MedCalc items, transfer recovery is 0.73 at layer 12, falls to 0.16 at 13, and stays below 0.08 from layer 15 onward. Skill-attention dependence is still 0.93 at layer 13 and 0.75 at 24, first falling to at most half at layer 25. Throughout layers 13–24, transfer recovers less than half the donor's gain while knockout loses more than half the originally correct answers. This is a separation of average intervention effects on common questions, not a claim that every question fails both interventions.

A stricter paired-outcome check keeps questions answered correctly by both protocols' donors and incorrectly by the receiver and the knockout protocol's no-skill baseline. On the resulting 124 Qwen3-8B/MedCalc questions, 100/124 (80.6%) fail both transfer and knockout at layer 13, rising to 113/124 (91.1%) at layer 16. Thus, the separation also occurs within individual questions. This diagnostic has a stricter screen than the main curves and retains their protocol differences (Appendix B).

The first crossing summarizes each curve. Qwen3-8B/MedCalc dependence briefly rises above half again at layer 26 before falling

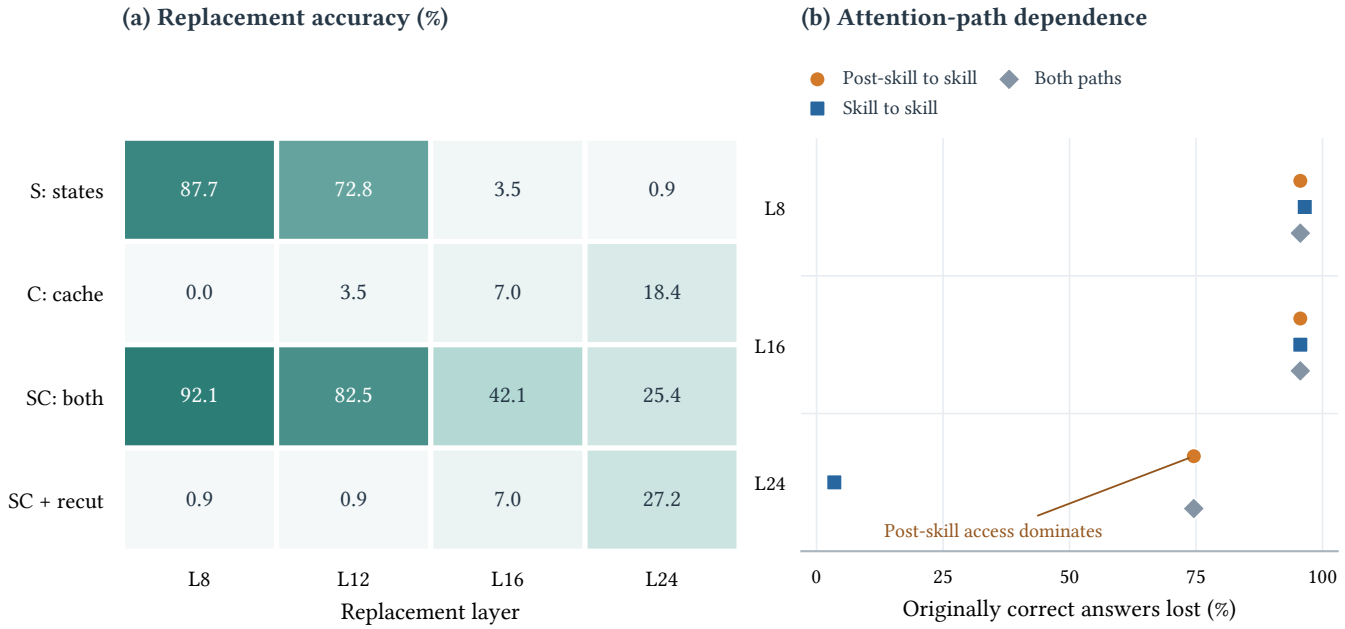


Figure 4: Skill-state/cache replacement and attention-path tests on the same 114 Qwen3-8B/MedCalc items from 14 skills. (a) Accuracy (%) following residual replacement (S), cache replacement (C), joint repair (SC), or joint repair followed by a post-skill-to-skill cut from the next layer. (b) Fraction of donor-correct answers lost when post-skill-to-skill, skill-to-skill, or both attention paths are cut from the indicated layer onward. The heatmap uses a fixed 0–100% scale with exact values in each cell. Path-cut estimates are offset vertically within each tested layer. Table 3 reports the main contrasts.

Table 1: Early transfer controls, Qwen3-8B/MedCalc, layer 8: 135 historically selected items from 14 calculators. Donor and receiver are re-evaluated under the transfer protocol; recovery uses the donor's 124 correct answers. Final-position patches replace fewer states than skill-span patches; other-skill content uses a matched prefix.

Intervention	Accuracy	ρ
Correct skill donor	.919	1.00
Wrong skill receiver	.000	.00
Correct skill span	.815	.89
Last prompt position	.030	.03
Last 16 prompt positions	.059	.06
Other skill content	.044	.05
Correct span, permuted	.044	.05
Norm-matched noise	.022	.02
Identity patch	.000	.00

below it from layer 27 onward. We retain these fluctuations in the measured curves. The measured interval summarizes weak transfer with strong aggregate skill dependence. The first crossing does not mean that transfer has vanished or that subsequent skill access is no longer needed.

Common-item and threshold checks. Original transfer and knockout runs can retain different eligible questions. We therefore recompute the curves over the intersection of their item IDs, using eligibility determined from unpatched baselines. Table 2 organizes four models by two benchmarks. All eight model–benchmark cells have verified depth results. On TheoremQA, Qwen3-8B has $T/R = 13/21$ (82 items, 66 skills), and Mistral-7B has 7/19 (72 items, 58 skills). The skill-dependence boundary follows the transfer boundary in every main comparison. For Llama-3.1-8B, T/R is 7/18 on MedCalc (112 items, 20 skills) and 6/18 on TheoremQA (92 items, 71 skills). For DeepSeek-LLM-7B, the corresponding boundaries are 6/21 (25 items, 13 skills) and 4/19 (54 items, 45 skills). Using transfer thresholds $\tau = 0.4$ or 0.6 and dependence thresholds $1 - \tau$ preserves the ordering in all eight main comparisons. This extends the observed separation to four model families, conditional on their respective baseline screens. The 25-item DeepSeek/MedCalc result supports only that selected population. Alongside sample and skill-group counts, Table 2 reports raw boundary differences and their percentage of model depth, $100\Delta/D$, where D is the number of layers. This normalization makes layer gaps comparable across model depths; it is not a benchmark-accuracy ranking. The supplementary Qwen3-0.6B comparisons on both tasks preserve the same ordering (Appendix B).

Broader screening and repeated replacement. With the less restrictive item-level receiver screen, 369 questions from 45 calculators yield recovery 0.94 at layer 8, 0.79 at 12 and 0.32 at 13. The early-to-late decline therefore persists outside the strict group screen.

Table 2: Main depth comparison across four open-weight models and two benchmarks. n/G gives common eligible questions/skill groups; T and R are the first half-effect crossings for transfer ($\rho < 0.5$) and aggregate skill dependence ($d \leq 0.5$). $\Delta = R - T$ is in layers; Δ/D normalizes the gap by the model's layer count. All eight cells are measured on their own baseline-screened, common-item populations.

Model	MedCalc-Bench					TheoremQA				
	n/G	T	R	Δ	Δ/D (%)	n/G	T	R	Δ	Δ/D (%)
Qwen3-8B	134/14	13	25	12	33.3	82/66	13	21	8	22.2
Mistral-7B	46/16	7	27	20	62.5	72/58	7	19	12	37.5
Llama-3.1-8B	112/20	7	18	11	34.4	92/71	6	18	12	37.5
DeepSeek-7B	25/13	6	21	15	50.0	54/45	4	19	15	50.0

This transfer-only check does not establish a full paired knockout curve under the broader screen. Nor does it remove the wrong-skill background; an equal-length neutral receiver is a separate control still needed.

Repeated full-span replacement produces the same outputs as a single write at the window's start in the audited comparison. Under causal attention and an identical preceding context, later writes can be redundant. We retain the window results as a consistency check in Appendix B, not as an independent exclusion of the single-write explanation.

4.4 Partial restoration with skill-cache replacement

Interventions and evaluation set. We compare S, C and SC as defined in Eqs. 1–2, plus SC followed by a post-skill-to-skill cut from layer $L+1$, denoted SC + recut. The depth separation motivates this test of a skill-position component omitted by residual replacement. It measures the effects of adding cache writes and of a subsequent post-skill-to-skill cut while retaining earlier computation at post-skill positions.

We evaluate layers 8, 12, 16, 24 on a fixed 124-question set from 14 calculators with fresh baselines. Of these, 114 questions are correct with the donor and incorrect with both the wrong-skill and no-skill baselines. Both panels of Figure 4 use these same questions. All 48 own-state controls across four GPU shards reproduce output tokens exactly. Appendix B.1 gives implementation and eligibility details.

Joint replacement gains. At layer 16, residual replacement answers 4/114 questions correctly, cache replacement answers 8/114, and joint repair answers 48/114. Table 3 reports the repair gain. The joint-minus-additive contrast is positive on the accuracy scale. It describes the tested conditions, but does not by itself identify a representation-level interaction or a unique computational mechanism [23]. At layer 24, joint repair still improves on residual replacement (25.4% versus 0.9%), compared with 18.4% for cache replacement alone. Adding skill-cache replacement therefore restores part of the deficit. Even the layer-16 joint condition remains 57.9 percentage points below the donor's 100% accuracy on these freshly screened items.

Table 3: Focused mechanism contrasts, 114 items from 14 skills. Differences are in percentage points (pp). Writing a_X for accuracy under condition X, the additive interaction is $a_{SC} - a_S - a_C + a_R$, where a_R is receiver accuracy.

Contrast	Difference
L16: $SC - S$	+38.6
L16: additive interaction	+31.6
L16: $SC - (SC$ + recut)	+35.1
L24: reading minus within-skill loss	+71.1

Continued reading and attention paths. Following joint repair at layer 16, cutting post-skill-to-skill attention reduces accuracy to 8/114. Most of the repaired effect therefore relies on continued reading. To identify what the original cumulative knockout disrupts, we separately cut post-skill-to-skill attention, skill-to-skill attention, or both in the unmodified donor, during prefill and decoding. At layer 24, post-skill-only and combined cuts each leave 29/114 questions correct, while a skill-to-skill-only cut leaves 110/114 correct. Post-skill-to-skill cuts account for most of the tested loss at this layer. Earlier cuts at layers 8 and 16 damage both paths. This decomposition covers three depths in one model–task setting; it cannot identify the post-skill-only component of all eight main curves. Because these cuts affect both prefill and decoding, they also do not locate the effect in either phase separately.

Depth variation and generation limits. After joint repair at layer 24, a post-skill-to-skill recut yields 27.2% accuracy, compared with 25.4% without the recut. The repaired-yet-reading-dependent result is therefore supported at layer 16 and varies across the tested depths. The main repair gain is stable to generation-length diagnostics: it remains 38.5 points on the 96 questions whose three baselines finish without truncation. An adverse reassignment of truncated outputs still leaves a 35.1-point gain (Appendix B.1).

Extension across models and tasks. Using the same split-prefill repair definitions and fresh baselines, we also evaluate the other seven main model–benchmark combinations. At the fixed layer 16, adding cache replacement to residual replacement improves accuracy in all eight settings, by 6.8–43.4 percentage points. A

next-layer post-skill-to-skill recut lowers joint-repair accuracy by 2.2–37.7 points. Table 5 reports every combination and its own denominator. These measurements extend the partial-repair observation, while their different effect sizes and remaining deficits do not establish a shared, sufficient mechanism.

What the repair does and does not identify. S changes a block's skill-position output and can thereby alter later skill states and caches; C writes skill-position caches at layers 0 through L. Combining them can increase coverage across the skill-cache stack. The observed gain is therefore compatible with a cache-coverage account, as well as with more specific interactions involving states and execution history. Discriminating these accounts requires all-layer skill-KV-only and later-layer KV-only controls with existing post-skill caches fixed, plus matched other-skill cache controls before claiming cache specificity. These conditions have not been measured here. The present result is a partial repair and a layer-specific recutting effect, not a complete or unique explanation of late transfer failure.

5 Conclusion

Weak skill-state transfer does not, by itself, show that the original execution has stopped depending on its skill text. Across eight completed main comparisons, residual transfer crosses below half of the donor's gain before aggregate skill-attention dependence falls to half. Early controls validate the transfer probe; common-item and threshold checks delimit the observed depth ordering. The result remains conditional on the screened populations and the wrong-skill receiver background.

The cache-replacement experiments supply a partial intervention result. At layer 16, joint replacement improves over residual replacement in all eight main settings; a next-layer post-skill-to-skill cut reduces that accuracy in each setting. The focused Qwen3-8B/MedCalc study shows a large repair and recutting effect. This makes skill caches one concrete contributor to test, while leaving cache coverage, post-skill-state changes and other omitted components unresolved. The recovery is incomplete, and the recutting effect varies with model, task and depth.

A stronger explanation requires all-layer and late-layer skill-cache controls, matched alternative-skill caches, and post-skill-only interventions that separate prompt processing from answer generation. Broader receiver backgrounds and model families can establish how far the present separation and partial repair extend to reusable skill systems.

References

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. doi:10.18653/v1/2026.acl-long.559

[3] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[5] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models? *arXiv preprint arXiv:2505.12075* (2025). https://arxiv.org/abs/2505.12075

[6] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[7] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[9] Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *arXiv preprint arXiv:2606.15017* (2026). https://arxiv.org/abs/2606.15017

[10] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[11] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[12] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[13] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[14] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[15] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. *arXiv preprint arXiv:2606.17107* (2026). https://arxiv.org/abs/2606.17107

[16] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[17] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[18] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[19] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract; arXiv:2511.10694.

[20] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.04510.

[21] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[22] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[23] Sankaran Vaidyanathan, David Arbour, Aaron Mueller, Scott Niekum, and David Jensen. 2026. The Curse of Multiple Mediators: Hidden Interaction Effects in Activation Patching. *arXiv preprint arXiv:2606.27510* (2026). https://arxiv.org/abs/2606.27510

[24] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. WebXSkill: Skill Learning for Autonomous Web Agents. *arXiv preprint arXiv:2604.13318* (2026). https://arxiv.org/abs/2604.13318

[25] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[26] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[27] Zhipeng Yang, Junzhuo Li, Siyu Xia, and Xuming Hu. 2025. Internal Chain-of-Thought: Empirical Evidence for Layer-wise Subtask Scheduling in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 22536–22564. doi:10.18653/v1/2025.emnlp-main.1147

[28] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[29] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A Experimental protocols and provenance

Tasks and scoring. The main MedCalc evaluation uses the 55 calculator skills released with SRA-Bench and 20 questions per calculator. The benchmark scorer checks free-form numerical answers against its prescribed tolerances. TheoremQA uses its released prompts, skill assignments and evaluator. All main experiments supply the selected skill directly and generate answers without tool calls. MedCalc questions are grouped by calculator; TheoremQA questions are grouped by skill identity.

Evaluation populations. The original candidate pool uses historical without-skill and with-skill behavioural outcomes: rescued means wrong then right, persistent failure means wrong in both, kept means right in both, and broken means right then wrong. The stored MedCalc screen contains 470 rescued IDs; the corrected transfer battery uses 467 from 48 calculators after its eligibility restrictions. The transfer code reads these frozen IDs and re-evaluates the donor, receiver and no-skill baselines under its own protocol.

The group screen filters on the re-evaluated receiver only: every sampled receiver answer in a retained skill group must be incorrect. It does not filter out re-evaluated donor failures. This gives 135 items in 14 groups, with 124 correct donor answers and 110 correct layer-8 span answers in Table 1. The denominator for recovery is the re-evaluated donor-minus-receiver gain, not historical donor accuracy. Selection and re-evaluation are separate operations; the present evidence does not attribute each change in correctness to one particular prompt, backend or numerical detail.

The depth comparison intersects this transfer population with the knockout run's own with-skill-correct, without-skill-incorrect IDs, retaining 134 questions. Transfer still uses its own donor and receiver outcomes, whereas knockout retention is normalized by its own correct-baseline set. Matching question IDs does not match the two execution protocols or make their outcomes complements.

The mechanism study freezes the same 124 IDs as the paired-outcome diagnostic (Appendix B) before new inference. All belong to the 134-item depth intersection, but their fresh split-prefill evaluation defines a separate population: 116 are donor-correct and receiver-incorrect, and 114 also fail without the skill. These 114 are shared by the repair and path-cut suites. The candidate count 124 and Table 1's 124 correct donor answers refer to different selection stages and should not be treated as interchangeable denominators.

The strict receiver screen selects a conditional effect; it does not eliminate interference from the wrong skill. Appendix B reports the broader item-level receiver screen. Neither screen substitutes for a position-matched neutral receiver intervention.

A.1 Implementation details

Models and decoding. The four main models are Qwen3-8B (36 layers, hidden width 4096), Mistral-7B-Instruct-v0.3 (32 layers, width 4096), Llama-3.1-8B-Instruct (32 layers) and DeepSeek-LLM-7B-Chat (30 layers). The supplementary Qwen3-0.6B check uses 28 layers and width 1024. The added Llama and DeepSeek depth runs use BF16, matched eager attention for full-prompt state capture and knockout, and 900 new tokens, without quantization. Regenerated Qwen and Mistral TheoremQA runs retain scaled dot-product attention for residual transfer and eager attention for knockout, with a 900-token limit for both. This is a fresh evaluation of frozen historical candidates; exact equivalence to the original knockout generation budget is not established. For DeepSeek, system instructions are merged into the user turn. The Llama weights, configuration and vocabulary match the original release; the public distribution supplies its tokenizer template. For Mistral, which has no system role in its chat template, system instructions are merged into the user turn. We use greedy decoding. Qwen thinking mode is disabled and checked on the rendered prompt. Attention-mask interventions use explicit attention computation with mask checks, often called eager attention. Earlier residual-replacement runs use PyTorch scaled dot-product attention. The mechanism experiments use BF16, a 16-bit floating-point format, and a 1,536-token generation limit.

Position and state checks. Donor and receiver prompts use length-matched skills, preserving question positions. Residual patches replace the selected block output during prefill. The captured positions are indexed within the prompt, separately from any answer tokens used for scoring. Identity controls write the receiver's own states back. A final-layer check also verifies that replacing a position's final hidden state reproduces the donor's next-token scores at that position. Attention cuts preserve the causal mask.

Focused mechanism protocol. A bridge on one fixed item from each of 14 calculators compares full-prompt prefill with prefill split before the final prompt token. All 84 own-state checks reproduce their corresponding receiver token sequences exactly. Changing the prefill protocol changes one donor's and one receiver's answer correctness. The formal repair and path comparisons therefore use the same split protocol and fresh no-skill, donor and receiver baselines. Truncation counts and sensitivity analyses are retained with the result records.

A.2 Evaluation scope and intervention provenance

Attention-mask correction. The original state-capture call omitted the attention mask supplied during decoding. Although mathematically equivalent for an unpadded sequence, these calls selected different numerical kernels. Their states differed by 1.2% at layer 8 and 25% at layer 34. Identity patching changed 108/1,766 Qwen3-8B outcomes (6.1%). Matching the masks restores exact identity agreement, including all 467 items in the corrected MedCalc rerun. All main-text state interventions use this correction. Attention knockout operates directly on attention masks and uses no donor-state capture.

Development and replication. The initial MedCalc intervention battery sampled four rescued questions from each of the 40 calculators with the most eligible questions. The corrected full-data rerun uses every rescued question from calculators with at least two, giving 467 items from 48 calculators. The main receiver screen then retains 135 questions from 14 calculators. Historical development results retain their original sample labels. Table 4 shows the corrected rerun alongside the earlier screened subset. The main depth

Table 4: Full-dataset rerun against the development subset, Qwen3-8B on MedCalc, layer 8. Recovery is Eq. 4; entries show point estimates. *per-calc* is the main text's restriction, *per-item* the weaker one. Controls reach 0.12 under the group screen and 0.25 under the item screen; each arm uses its own matched baselines.

arm	full, per-calc (n = 135)	subset (n = 60)	full, per-item (n = 369)
transplant `real`	**0.89**	0.87	0.94
same skill, matched prefix	0.65	0.73	0.84
half dose $\alpha = 0.5$	0.04	0.06	0.11
positions permuted	0.05	0.08	0.21
norm-matched noise	0.02	0.10	0.18
other calculator	0.05	—	0.25
other family, mean	0.05	0.06	0.16
same family, mean	0.12	0.05	0.24
identity patch	0.00	0.00	0.00
rank 128 / 64 / 32 / 16	0.77/0.63/0.12/0.05	0.77/0.54/0.13/0.06	0.88/0.70/0.22/0.18
bottom-64 / bottom-16	0.03/0.04	0.00/0.06	0.11/0.12
quarters $q_0/q_1/q_2/q_3$	0.09/**0.56**/0.12/0.05	0.12/0.56/0.12/0.12	0.18/0.64/0.22/0.19

Table 5: State/cache repair at the fixed layer 16 on all main model–task combinations. Accuracies are percentages. Gain is $a_{SC} - a_S$; recut loss is $a_{SC} - a_{SC+\text{recut}}$, both in percentage points. Each row uses its own n questions from G skill groups.

Model	Task	n/G	S	C	SC	SC+recut	Gain (pp)	Recut loss (pp)
Qwen3-8B	MedCalc	114/14	3.5	7.0	42.1	7.0	38.6	35.1
Qwen3-8B	TheoremQA	53/43	18.9	15.1	62.3	24.5	43.4	37.7
Mistral-7B	MedCalc	38/15	7.9	18.4	18.4	5.3	10.5	13.2
Mistral-7B	TheoremQA	59/49	3.4	5.1	10.2	6.8	6.8	3.4
Llama-3.1-8B	MedCalc	104/20	3.8	5.8	18.3	9.6	14.4	8.7
Llama-3.1-8B	TheoremQA	72/56	4.2	12.5	41.7	12.5	37.5	29.2
DeepSeek-7B	MedCalc	21/10	0.0	19.0	28.6	19.0	28.6	9.5
DeepSeek-7B	TheoremQA	46/40	2.2	6.5	10.9	8.7	8.7	2.2

study covers all eight model–task comparisons across four models and families. Qwen3-0.6B supplies two supplementary comparisons, one per task. Each comparison uses common eligible items for the two intervention types. The new-family candidate pools inherit the earlier models' selection, so their results are conditional rather than full-benchmark estimates.

Extended control populations. Table 6 records the independent populations for content/position, fixed layer-2 sensitivity, skill-after-question, repeated-write, cache-repair and attention-path experiments. We do not transfer the depth comparison's denominator to another protocol. For TheoremQA, all retained text is scored with fresh symbolic-parser state per answer, checked in reverse order and against fresh processes. The corrected DeepSeek depth population contains 54 items; repair and path-cut protocols independently retain 46 of those candidates from 40 skills. Their three baseline token sequences agree on all 54 candidates. Cross-question tests require another question using the same skill, and same-family controls require another available skill in that family. Unavailable donors are excluded from the corresponding arm's population rather than assigned zero effect. The two-candidate bridge checks validate execution timing and are not effect estimates over an entire model–benchmark population.

Scientific scope. The experiments characterize supplied skill instructions under fixed selection and controlled generation. The depth ordering holds for the reported conditional populations and protocols. Historical transfer and knockout differ in backend; the added families use matching eager-attention protocols. The cache-repair tests include all eight main combinations, with independently screened populations. The detailed path decomposition and generation-length sensitivity discussion focus on Qwen3-8B/MedCalc. All-query skill knockout combines access from post-skill tokens with skill-state updates; its full-depth profile cannot be interpreted as post-skill-only dependence. Prefill and decoding are jointly intervened on, so phase-specific claims require separate tests. The stored paired-outcome diagnostic supports same-question joint failures in three MedCalc settings; it does not extend that evidence to all completed comparisons. Interactive

Table 6: Independent eligible question counts for the four main models. Cross-question counts require an available other-question donor. The Qwen3-8B/MedCalc layer-8 battery retains its historical group screen; other rows apply their own unpatched baselines. Within a battery, variable-donor controls can have smaller populations.

Model	Task	L8 battery	L2 battery	Cross-question	Windows	Repair	Path cuts
Qwen3-8B	MedCalc	135	97	151	135	114	114
Qwen3-8B	TheoremQA	57	61	8	61	53	53
Mistral-7B	MedCalc	35	35	10	35	38	38
Mistral-7B	TheoremQA	61	61	7	61	59	59
Llama-3.1-8B	MedCalc	112	112	92	112	104	104
Llama-3.1-8B	TheoremQA	92	92	46	92	72	72
DeepSeek-7B	MedCalc	25	25	25	25	21	21
DeepSeek-7B	TheoremQA	54	54	23	54	46	46

browser tasks, skill retrieval errors and learned state compression are natural settings in which to test how these findings extend to agent systems.

B Additional transfer and skill-dependence analyses

The initial Qwen3-8B/MedCalc battery sampled four historically rescued questions from each of 40 calculators (160 candidates), retaining 60 questions from 15 calculators under the receiver-never-solves screen. The corrected rerun evaluates all 467 candidates from 48 calculators and retains 135 questions from 14 calculators. A weaker item-level receiver screen retains 369 questions from 45 calculators. The group screen becomes stricter as more questions from each calculator are tested. Table 4 compares the fixed interventions under these screens.

For the dose controls, $\delta^L = H_D^L[\mathcal{I}] - H_R^L[\mathcal{I}]$ and the write is $H_R^L[\mathcal{I}] + \alpha\delta^L$. Rank truncation retains the leading k singular directions of this difference; bottom-k retains the least prominent. The formula quarter provides the largest standalone recovery in both samples. Same-family means use their available-donor population (41 items in the full rerun).

One-layer resolution. The initial depth grid could only place the decline after layer 12. The completed grid measures every layer. Under the group screen, recovery falls from 0.74 at 12 to 0.16 at 13; the item screen gives 0.79 and 0.32. Both screens have their largest local decline at this same step, while Figure 3 retains the later fluctuations rather than imposing monotonicity.

Supplementary model-scale check. Qwen3-0.6B is retained as a supplementary within-family scale comparison, rather than a fifth model in the main comparison matrix. Its MedCalc common-item set contains 90 questions from 21 skills. The half-effect boundaries are $T = 11$ and $R = 22$, giving $\Delta = 11$ layers, or 39.3% of its 28-layer depth. Transfer/dependence boundary pairs are 11/21 at threshold 0.4 and 11/22 at both 0.5 and 0.6. On TheoremQA, fresh isolated grading and independent baseline screening retain 122 common questions from 92 skills, with $T/R = 11/20$ ($\Delta = 9$, or 32.1% of model depth). The corresponding threshold pairs are 11/20, 11/20 and 11/21 at 0.4, 0.5 and 0.6. Together with the eight main comparisons,

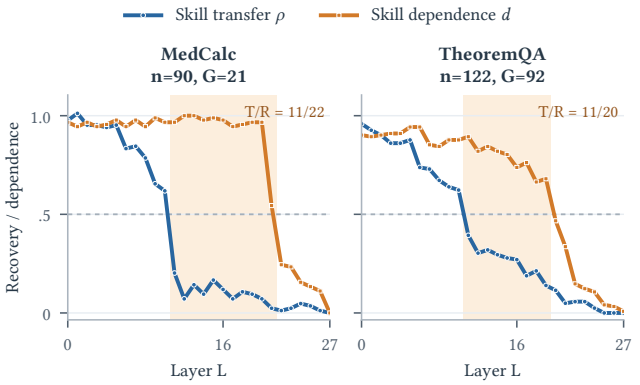


Figure 5: Supplementary Qwen3-0.6B depth profiles on both benchmarks. Each panel uses common eligible items for transfer and knockout, with each protocol's own baselines. Curves, first half-effect boundaries and skill-group counts use the same definitions as Figure 3.

these give ten measured comparisons across five model sizes from four families. Figure 5 shows both supplementary curves.

Same-question joint failures. Stored per-skill paired-outcome summaries retain questions with a correct donor under both protocols, an incorrect transfer receiver and an incorrect no-skill baseline under knockout. Weighting each group's joint failure fraction by its item count gives 100/124 (80.6%) for Qwen3-8B at its transfer boundary $T = 13$, 64/84 (76.2%) for Qwen3-0.6B at $T = 11$, and 25/43 (58.1%) for Mistral-7B at $T = 7$. These MedCalc sets contain 14, 20 and 15 skill groups, respectively. Their stricter donor screen differs from the populations in Table 2. The diagnostic also retains protocol mismatch: 4, 0 and 4 items, respectively, are correct under the transfer protocol's no-skill baseline despite failing that baseline under knockout. These are paired outcomes under two interventions, not a claim of identical baseline executions. We report this stricter paired-outcome diagnostic for these three historical MedCalc settings only; common IDs alone do not establish that every question fails both interventions elsewhere.

Repeated replacement as a consistency check. On the 135 historically screened MedCalc items, replacing skill states over windows 8–11, 12–15, 14–19 and 16–35 recovers 0.89, 0.74, 0.16 and 0.06, respectively. Replacement from the first layer onward recovers 0.98. The recorded repeated-write and same-start single-write checks found identical output tokens. With identical preceding context and causal attention, a full skill-span write can already determine later skill states, making later writes redundant. These measurements do not independently rule out inadequate state maintenance or identify a cause of the late transfer deficit.

A continuous readout at the same boundary. Accuracy thresholds a generated answer, so an internal change can appear sharper in this measure. We therefore generate the correct-skill reasoning once per item and score that fixed sequence under each intervention. This is teacher forcing: the model receives the recorded preceding tokens rather than generating a new sequence. The mean token log-probability measures how strongly it predicts that reasoning. Log-probabilities use natural logarithms, with units called nats. Higher values indicate that the recorded sequence is more likely.

We normalize this score as $z = (\ell_L - \ell_{\text{recv}})/(\ell_{\text{gold}} - \ell_{\text{recv}})$, where ℓ denotes the mean log-probability and the subscripts identify the patched, receiver and donor conditions. We use z to distinguish this diagnostic from knockout retention q. On 442 donor-correct items from 48 calculators, z is 0.999, 0.989, 0.960, 0.831 at layers 0, 4, 8, 12. It then falls to 0.598 at layer 13, 0.595 at 14, 0.340 at 16, 0.201 at 20, and 0.129 at 24. The layer-12 to layer-13 decline spans 26.8% of the score's range, compared with the 61% behavioural decline. The continuous readout therefore also changes sharply at the accuracy boundary.

The decline is distributed across the four quarters of the reasoning sequence (0.20, 0.26, 0.31, 0.21). Scoring only tokens after the final answer marker is less useful here. Once the reasoning is supplied, all conditions predict the final number closely: median log-probabilities lie between −0.001 and −0.005, and the donor-minus-receiver denominator has median 0.0014, below 0.01 on 59% of items.

Distributed reading and receiver state. An attention head is one of the parallel attention components within a transformer block. Head-level readout from the skill span concentrates in layers 12–21 after normalisation by residual-stream norm. Blocking the 32 highest-ranked heads costs 0.39 of the skill's effect, versus 0.23 for random heads at the same layers (held-out calculators; paired difference 0.154). Reading is distributed rather than confined to a small circuit. After late span replacement, downstream readout remains 0.86–0.99 of the donor's despite weak behavioural recovery. Earlier receiver question-state updates remain unchanged, motivating the interaction tests in §4.4.

B.1 Focused skill-cache and path-specific experiments

The focused Qwen3-8B/MedCalc experiments fix 124 historical item IDs from 14 calculators before inference and partition them as every fourth ID across four RTX 4090 workers. They use BF16, eager attention, greedy decoding and 1,536 new tokens. Fresh baselines retain 116 donor-correct, receiver-incorrect items, of which 114 also fail without a skill. Both experiments use these same 114 items; all 372 baseline generations agree token-for-token between the two suites. The complete repair grid contains 2,356 generations, the path grid 1,488, and worker identity checks 100. All 48 own-state writebacks are token-exact.

S writes the donor skill-span block output at layer L. C replaces only skill-position keys and values at layers 0 through L after prefix prefill and before recomputing the final prompt token. SC combines them; SC recutting blocks post-skill queries to skill keys from $L+1$ onward in that same run. Earlier computation at post-skill positions is unchanged. Post-skill queries are positions after the skill span; within-skill queries are positions within it. The path experiments cut the specified edges during both prefill and decoding, preserving the causal mask.

This timing matters for interpretation. State replacement can propagate into later skill-position caches and later computation at post-skill positions, while the explicit cache write covers layers 0 through L. The joint intervention changes both components and their coverage, so its accuracy interaction does not uniquely identify a residual–history compatibility mechanism. An all-layer skill-KV-only condition, a later-layer KV-only condition and matched other-skill cache replacements are needed to distinguish these possibilities. These controls are not part of the reported grid.

We retain all intervention outcomes, including truncated outputs. For the prespecified SC-minus-S contrast at layer 16, the gain is 38.6 percentage points. The corresponding joint-minus-additive contrast is 31.6 points; SC-minus-recut is 35.1 points. On the 96 items with all three baselines untruncated, SC-minus-S is 38.5 points. At layer 16, S/SC/recut truncate 4/4/3 of 114 outputs; worst-case reassignment of all truncated S and SC outcomes leaves a 35.1-point repair advantage. At layer 24, SC and recut truncate 3/28 outputs and show no additional loss in measured accuracy; this depth is not used to claim independence after repair.

At layer 24, post-skill and skill-to-skill cuts retain 29/114 and 110/114 correct answers. Their loss difference is 71.1 points. Reader cuts truncate 21 outputs; counting all as correct still leaves retention at 50/114, below skill-to-skill retention 110/114 (no truncation). Both contrasts survive worst-case reassignment of truncated outputs. Analysis artifacts retain arm-wise counts, hashes and truncation bounds.

Repair extension across families and benchmarks. Table 5 reports the same fixed layer-16 contrast on all eight main combinations. Every row uses its own fresh split-prefill baselines, retaining donor-correct questions that both the wrong-skill receiver and no-skill execution answer incorrectly. Repair and path-cut baseline token sequences match within each model–task combination. The extension keeps the 1,536-token limit and includes truncated generations in the accuracy denominator. The control and mechanism populations differ from the full-prefill depth populations; for example, Qwen3-8B/TheoremQA has 82 depth items and 53 repair items. The main text's truncation sensitivity bounds apply to the focused Qwen3-8B/MedCalc study, not automatically to every extension row.